

Q. * What is TLB? / Briefly explain the function of TLB.

Solⁿ:

TLB (Translation Lookaside Buffer) is a tiny, special cache inside the processor that stores recent address translations.

Virtual memory requires checking the page table in RAM before every memory access, which slows the system down.

It acts as a speed cheat sheet. When the CPU needs a physical address, it checks the TLB first. If it's a TLB hit, the CPU gets an instant physical address without checking the slow RAM.

The main page table is like a huge phonebook (slow to search), while the TLB is like your frequently called contacts list (instant access).

* Working principle of TLB with figures/Flowchart / How TLB improves memory access time.

Soln:

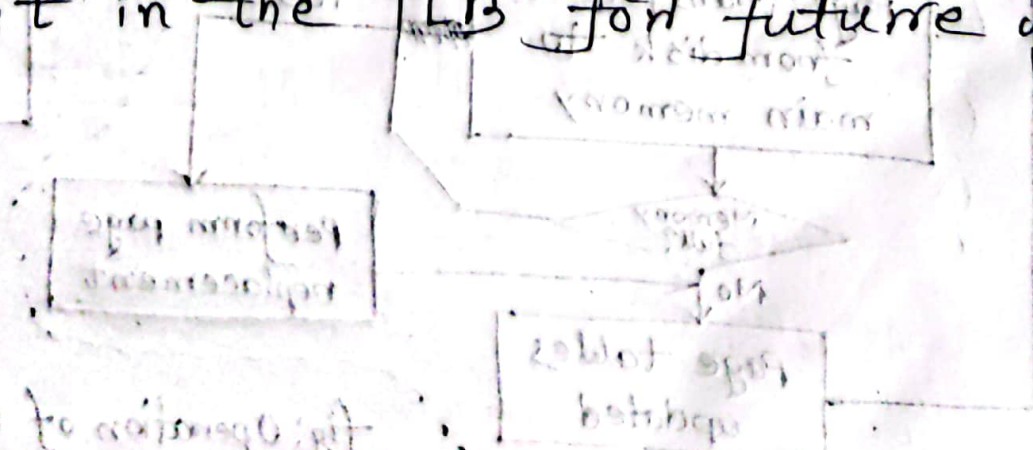
1. Working Principle (How it works & Improves Time)

⇒ The CPU generates a virtual Address to look up data.

⇒ (TLB Check): Before checking the slow RAM, the CPU looks inside the TLB.

⇒ (TLB Hit): If the translation is found in the TLB, it instantly provides the Physical Address. This avoids doubling the memory access time and speeds up the process.

⇒ (TLB Miss): If it's not in the TLB, the CPU must look up the main Page Table in RAM, fetch the translation, and then save it in the TLB for future use.



2. Flowchart & Operations

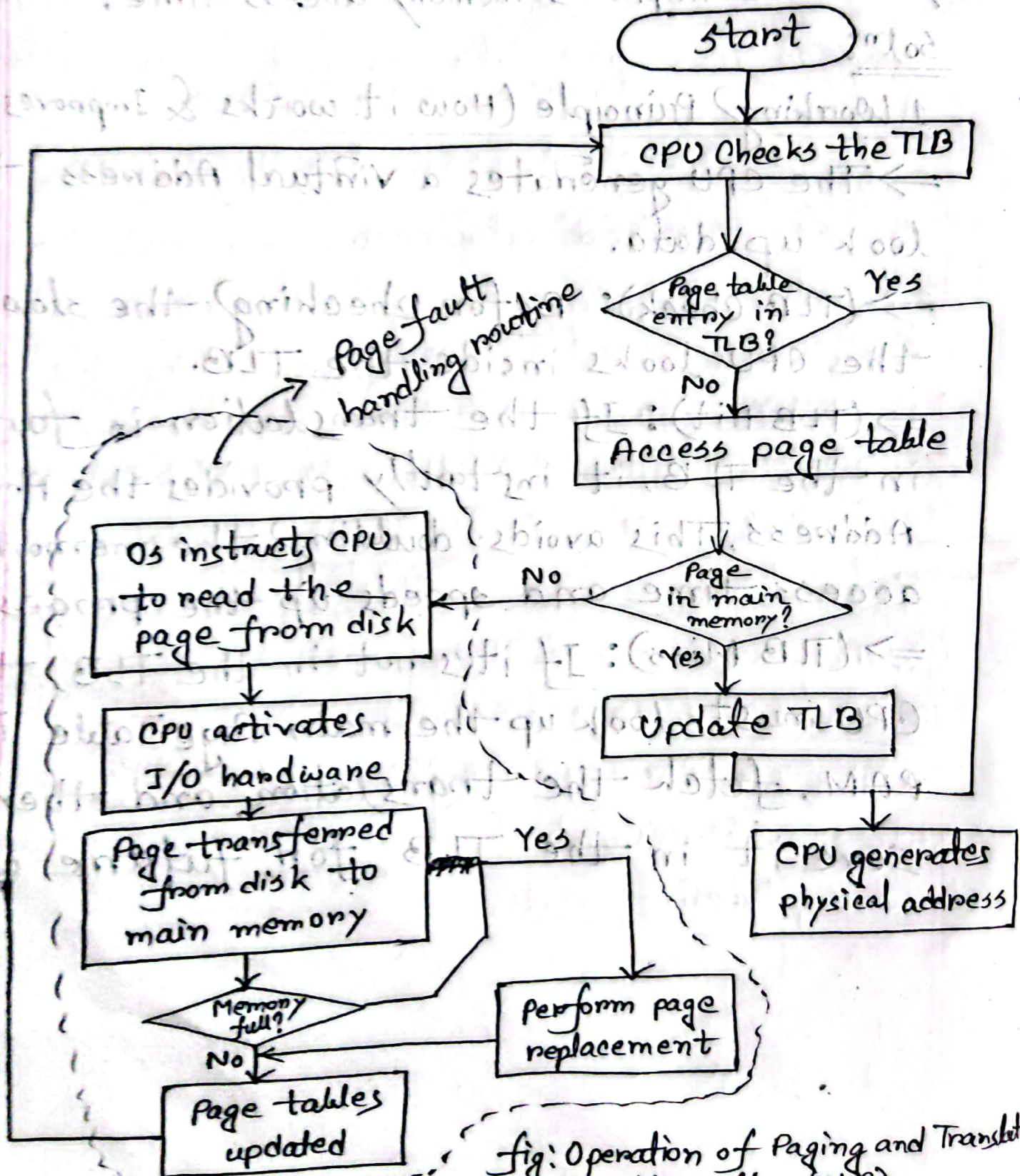


Fig: Operation of Paging and Translation using Lookaside Buffer (TLB).

* What happens on a TLB miss?

Soln:

A TLB miss occurs when the address translation requested by the CPU is not present in the TLB cache.

Step-by-step process:

1. Page Table Lookup: The CPU must take a slower path and search the main Page Table located in RAM.

2. Check for Page Presence:

→ If page is in Memory (Page Hit): The translation is retrieved from RAM, the TLB is updated with this new entry, and the CPU generates the physical address.

→ If Page is NOT in Memory (Page Fault): The system triggers a Page Fault Handling Routine. The OS instructs the CPU to fetch the missing page from the disk into main memory, updates the page Table, and restarts the instruction.